C++中 char 字符数组与常用字符串函数详解

我来用**最通俗、最贴合新手**的方式，给你讲清楚 C++ 中 **char 字符数组（C 风格字符串）**、结束符 `\0`，以及 `strlen/strcpy/strcat/strcmp` 这 4 个最常用的字符串函数。

一、核心前提：char 数组与 `\0` 结束符

在 C++ 中，**用 char 数组存储字符串（C 风格字符串），必须以 `\0`（空字符，ASCII 码为 0）结尾。**

这是所有字符串函数的**唯一依据**——函数通过找 `\0` 来判断字符串在哪里结束。

1. 正确定义 char 字符串数组

Code block

```
// 写法1：双引号赋值 → 自动在末尾加 \0（最常用）
char str1[10] = "hello";
// 实际存储：h e l l o \0 （后面3个空间空闲）
// 有效字符5个，总数组大小10

// 写法2：手动定义 → 必须自己写 \0
char str2[6] = {'h','e','l','l','o','\0'};
```

2. 错误写法（千万不要）

Code block

```
// 没有 \0！字符串函数会一直往后读内存，导致乱码、程序崩溃
char str[5] = {'h','e','l','l','o'};
```

二、必备头文件

所有字符串函数都需要这个头文件：

Code block

```
#include <cstring>  // C++ 推荐写法
// 或 #include <string.h> （C 语言老式写法）
```

三、四大常用函数逐行讲解

我会用 **作用 + 语法 + 示例 + 关键点** 的方式，简单直接。

1. strlen：计算字符串有效长度

作用

计算字符串的**实际字符个数（不算 `\0`）**。

语法

Code block

```
size_t strlen(const char* str);
```

示例

Code block

```
char str[] = "hello";
cout << strlen(str);  // 输出 5（只算 h e l l o）
```

核心区别

- `strlen(str)` → 有效字符长度（无 `\0`）
- `sizeof(str)` → 数组总占用空间（含 `\0`）

2. strcpy：字符串拷贝

作用

把**源字符串**完整复制到**目标数组**中（**连 `\0` 一起拷贝**）。

语法

Code block

```
char* strcpy(char* 目标数组, const char* 源字符串);
```

示例

Code block

```
char dest[20];              // 目标数组（空间要足够大）
char src[] = "hello world";
strcpy(dest, src);          // 把 src 拷贝到 dest
cout << dest;               // 输出: hello world
```

⚠ 关键警告

目标数组空间必须 ≥ 源字符串长度 + 1（留位置给 `\0`），否则会**内存溢出**！

3. strcat：字符串拼接

作用

把**源字符串**拼接到**目标字符串的末尾**，覆盖目标原来的 `\0`，最后自动补新的 `\0`。

语法

Code block

```
char* strcat(char* 目标数组, const char* 源字符串);
```

示例

Code block

```
char str[20] = "hello";  // 目标数组必须预留足够空间
strcat(str, " c++");     // 拼接
cout << str;             // 输出: hello c++
```

⚠ 关键警告

1. 目标数组空间必须足够大（容纳拼接后的总长度）
2. 目标字符串必须以 `\0` 结尾

4. strcmp：字符串比较

作用

按 ASCII 码逐字符比较两个字符串（**区分大小写**）。

语法

Code block

```
int strcmp(const char* str1, const char* str2);
```

返回值规则（死记这 3 个）

返回值	含义
0	两个字符串**完全相等**
负数	str1 **小于** str2
正数	str1 **大于** str2

示例

Code block

```
cout << strcmp("apple", "banana");  // 负数（a < b）
cout << strcmp("abc", "abc");       // 0（相等）
cout << strcmp("Hello", "hello");   // 负数（H(72) < h(104)）
```

四、完整可运行代码（整合所有函数）

直接复制运行，一看就懂：

Code block

```
#include <iostream>
#include <cstring>  // 字符串函数必备
using namespace std;

int main() {
    // 1. 定义 char 字符串（自动带 \0）
    char str1[20] = "hello";
```

```
    char str2[20] = "world";
    char dest[50];  // 用于拷贝、拼接的目标数组

    // 2. strlen: 计算长度
    cout << "strlen(str1) = " << strlen(str1) << endl;  // 5

    // 3. strcpy: 拷贝字符串
    strcpy(dest, str1);
    cout << "拷贝后 dest = " << dest << endl;  // hello

    // 4. strcat: 拼接字符串
    strcat(dest, " ");
    strcat(dest, str2);
    cout << "拼接后 dest = " << dest << endl;  // hello world

    // 5. strcmp: 比较字符串
    cout << "strcmp(str1, str2) = " << strcmp(str1, str2) << endl;  // 负数
    cout << "strcmp(\"abc\", \"abc\") = " << strcmp("abc", "abc") << endl;  // 0

    return 0;
}
```

运行结果

Code block

```
strlen(str1) = 5
拷贝后 dest = hello
拼接后 dest = hello world
strcmp(str1, str2) = -15
strcmp("abc", "abc") = 0
```

五、新手必记 4 条铁律

1. **`\0` 是灵魂**：所有 char 字符串必须以 `\0` 结尾，否则函数全部失效。
2. **`new[]/delete[]` 配套**：如果用 `new char[len]` 创建数组，记得手动加 `\0`，释放用 `delete[]`。
3. **空间要足够**：`strcpy/strcat` 必须保证目标数组够大，防止溢出。
4. **区分大小写**：`strcmp` 严格按 ASCII 码比较，大写字母 < 小写字母。

总结

1. **char 数组 +** `\0` = C 风格字符串
2. **strlen**：算有效长度（无 `\0`）
3. **strcpy**：拷贝字符串
4. **strcat**：拼接字符串
5. **strcmp**：比较字符串（返 0/正/负）

（注：文档部分内容可能由 AI 生成）